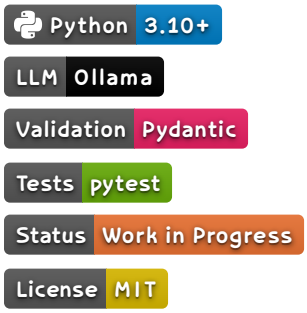
🛡️ incident-guard

> Don't trust the AI blindly. Validate it. ✅

A beginner-friendly Python tool that takes an incident report written in plain English, asks a local AI model to extract the key details, and then checks the AI's answer for missing, wrong, or invented information.



🤔 Why does this project exist?

AI models are great at reading messy text, but they sometimes:

- 🕳️ **Leave fields empty without telling you**
- 📅 **Return impossible dates like `31 February`**
- 📞 **Produce broken phone numbers**
- 👻 **Invent details that were never in the text (hallucinations)**

In serious situations, like emergency-response paperwork, a silent mistake can cause real trouble. `incident-guard` sits between the AI and your final output and reports exactly what is wrong instead of quietly accepting it.

💡 Inspiration

This project is inspired by FireForm, an open-source "report once, file everywhere" system for first responders. FireForm extracts incident data using a local LLM; `incident-guard` is my own small project exploring the "can we trust that extraction?" part of the problem.

> ⚠️ This is an independent learning project. It is not affiliated with FireForm.

⚙️ How it works

```
📝 Incident narrative (plain text)
          |
          ▼
🤖 Local LLM via Ollama   →   raw JSON
          |
          ▼
🛡️ Pydantic schema validation
          |
```

```
                ▼
📋 Report: ✅ valid fields | ❌ missing | ⚠️ invalid | 👻 unexpected
```

✨ Planned Features

- 🔌 **Talk to a local Ollama model (no internet or paid API needed)**
- 🧱 **Define the expected fields using a Pydantic schema**
- 🔍 **Detect missing required fields**
- 📅 **Detect invalid dates and malformed phone numbers**
- 👻 **Detect unexpected keys the model made up**
- 🧪 **A test suite with fake incident narratives**

🗺️ Roadmap

I'm building this in small stages so each one works on its own.

- **[ ] Stage 1: Extract → send a narrative to Ollama and print the JSON**
- **[ ] Stage 2: Schema → describe expected fields with Pydantic**
- **[ ] Stage 3: Validate → report missing, invalid, and invented fields**
- **[ ] Stage 4: Test → write 10–15 fake narratives and `pytest` tests**

📦 Installation

> 🚧 Setup steps will be finalized as the code is written. The plan:

```
# 1. Clone the repo
git clone https://github.com/<your-username>/incident-guard.git
cd incident-guard

# 2. Create a virtual environment
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Install Ollama and pull a small model
ollama pull llama3.2:3b
```

💻 **Low on RAM? Try a smaller model such as `llama3.2:1b`.**

🚀 Usage (planned)

```
python main.py "At 2:30 PM on 12 March, a brush fire was reported near Pune Road. Two engines
responded. No injuries."
```

Example output:

```
✅ incident_date: 12 March
✅ location: Pune Road
✅ units_responded: 2
⚠️ incident_time: could not be parsed
❌ reporting_officer: missing
```

🗂️ Project Structure (planned)

```
incident-guard/
├── main.py            # entry point
├── extractor.py       # talks to Ollama
├── schema.py          # Pydantic models
├── validator.py       # validation logic
├── data/              # fake incident narratives
├── tests/             # pytest tests
├── requirements.txt
└── README.md
```

🧰 Tech Stack

Tool	Purpose
🐍 Python	Core language
🦙 Ollama	Runs the LLM locally and for free
🧱 Pydantic	Schema and validation
🧪 pytest	Automated testing
🌿 Git & GitHub	Version control

🎯 What I'm learning

- Working with local LLMs and structured output
- Data validation with Pydantic
- Writing tests with pytest
- Handling edge cases and errors
- Clean Git workflow and documentation

🤝 Contributing

Suggestions and ideas are welcome! Open an issue to discuss a change before sending a pull request. 💬

📄 License

Released under the MIT License.

⭐ If you find this project interesting, consider giving it a star!